

AI-Based Clinical Decision Support System

DBMS Projects – Architecture & Frontend Overview

Prof. ACS Rao

Associate Professor
Indian Institute of Technology (Indian School of Mines)

Project Objective

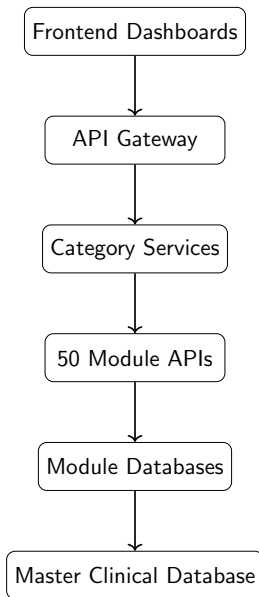
- Build a **Database-Centric Clinical Decision Support System**
- Focus strictly on **core DBMS concepts**
- No Machine Learning – logic via:
 - SQL Queries
 - Views
 - Triggers
 - Stored Procedures
- Large-scale system will be developed by **50 independent groups with 3 students in each group.**
- The details of the project are attached at last.

Overall System Philosophy

Key Design Idea

- Modular Architecture
- Each team develops one independent module
- Modules are grouped into Categories (A–I)
- Categories connect to a Master Dashboard
- All data integrates into a Master Database

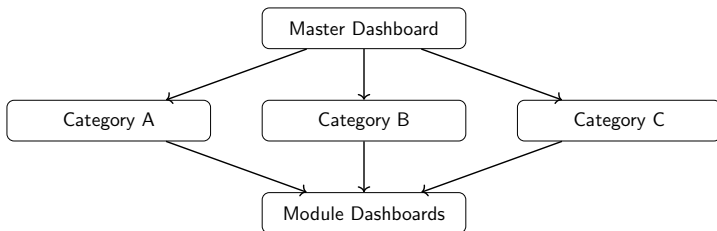
System Architecture Overview



Frontend is divided into **3 hierarchical levels**

- 1 Master Dashboard
- 2 Category Dashboards
- 3 Module Dashboards

Frontend Hierarchy Diagram



Master Dashboard

Purpose:

- Overall system view
- Used for demonstration and evaluation

Features:

- Category-wise statistics
- Global patient index
- System-wide alerts
- Audit logs

Category Dashboards

Each Category (A–I) has:

- Overview metrics
- List of assigned modules
- Aggregated SQL views
- Rule-based alerts

Example Categories:

- Patient Data Management
- Drug Safety Systems
- ICU Monitoring
- Laboratory Interpretation

Module Dashboards (Team Responsibility)

Each Team Develops:

- One independent module
- One database schema
- One backend API
- One frontend dashboard

Module UI Must Show:

- Tables
- SQL Queries
- Triggers / Procedures
- Output results

Frontend Module Contribution Guidelines (Students)

Repository Access:

- Work using a FORK of the main repository
- No direct access to organization repository

Team Setup (3 students):

- One member forks the repository
- Others added as collaborators to the fork

Mandatory Workflow:

- 1 Fork organization repository
- 2 Clone your fork locally
- 3 Create branch: `module/<module-name>`

Frontend Module Contribution Guidelines (Contd.)

Folder Rule:

- Work ONLY inside: `src/modules/<module-name>/`
- Do NOT modify other modules without approval

Commits:

- ALL team members MUST make commits
- Meaningful commit messages required
- Good: "Add login form UI"
- Bad: "final", "update", "done"

Push & Pull Request:

- Push module branch to your fork
- Create PR to organization repository
- One module = One Pull Request

Frontend Module Contribution Guidelines (Contd.)

PR Requirements:

- Module name
- Team member names + GitHub IDs
- List of features implemented
- Individual contribution summary

Resources:

- **GitHub Organization:**

<https://github.com/orgs/Medical-Copilot-System/repositories>

- **Tutorial Video for Github Tools:**

<https://youtu.be/q8EevlEpQ2A>

- **Tutorial Documentation to Connect Streamlit to Mongo DB:**

<https://docs.streamlit.io/develop/tutorials/databases/mongodb>

<https://docs.streamlit.io/develop/tutorials/databases/mongodb>

Frontend Module Evaluation Guidelines (Teachers)

Submission Format:

- All work via Pull Requests (PRs)
- Pull Requests = Assignments

Primary Evaluation Tool:

- Files changed
- Lines added/removed
- Commit history
- Individual contributors

Checkpoints for Evaluation:

- File changes restricted to assigned module
- All team members have commits
- Commit quality and messages

Frontend Module Evaluation Guidelines (Contd.)

Contribution Verification:

- Use PR → Commits to assess individual effort
- Cross-check PR description with commit history
- Filter commits by author if needed

Cheating/Free-riding Indicators:

- Only one student committing
- Identical code across teams
- Last-minute bulk commits
- Copy-paste patterns

Feedback Method:

- Use PR comments for technical feedback
- Request changes if necessary
- Approve PR only after verification

Frontend Module Evaluation Guidelines (Contd.)

Recommended Grading Rubric:

- Code correctness & completeness: 30%
- Individual contribution (commits): 25%
- Pull Request quality: 20%
- Folder & module discipline: 15%
- Commit hygiene & clarity: 10%

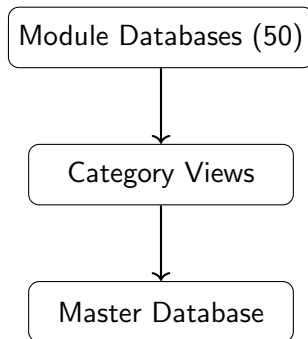
Record Keeping:

- PR links as submission proof
- GitHub history as plagiarism-safe evidence

Optional Enhancements:

- Protect main branch
- Use GitHub Insights → Contributors

Backend Database Architecture



Master Database Concept

Master DB Contains:

- Unified patient registry
- Cross-module views
- Global alerts
- Audit and access logs

Populated Using:

- SQL Views
- Triggers
- Stored Procedures

Integration Strategy

- 1 Each team completes module independently
- 2 APIs validated at category level
- 3 Data exposed via controlled views
- 4 Final integration into Master Database

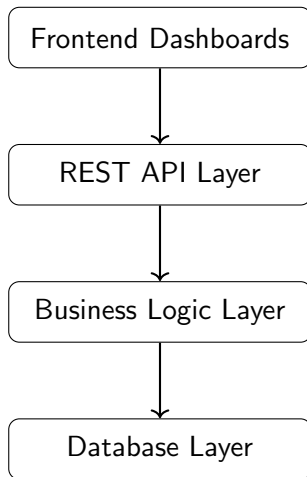
What Will Be Evaluated

- Database design and normalization
- Use of constraints
- Quality of SQL queries
- Triggers and procedures
- Integration correctness
- Understanding during viva

Backend Architecture Overview

- Backend follows a **modular service-oriented architecture**
- Each module has:
 - Its own database
 - Its own backend service (API)
- Category services aggregate multiple modules
- Master backend connects all categories

Backend Layered Architecture



Module-Level Backend Responsibility

Each Team Must Implement:

- One backend service (API)
- Database connection logic
- CRUD operations using SQL
- Triggers / Procedures in DB

Important Rule:

Isolation Principle

Modules must NOT directly access other module databases.

Why APIs?

- To allow controlled communication between modules
- To enable category-level aggregation
- To support master dashboard integration

API Types:

- Module APIs
- Category APIs
- Master APIs

API Communication Flow



Integration is done using DBMS concepts only:

- SQL Views
- Stored Procedures
- Triggers

Flow:

- 1 Module DB generates validated data
- 2 Exposed through views or procedures
- 3 Pulled into Master DB

Role of Master Database

- Acts as the single source of truth
- Maintains global patient and visit indexes
- Enables cross-category queries
- Supports master dashboard analytics

What You Must Learn – Backend Concepts

- Client–Server Architecture
- RESTful APIs (GET, POST, PUT, DELETE)
- Database Connectivity (JDBC / Python / PHP)
- API Testing (Postman / curl)

What You Must Learn – DBMS Concepts

- ER Modeling and Normalization
- Constraints and Referential Integrity
- SQL Queries (Basic to Advanced)
- Views and Materialized Views
- Triggers and Stored Procedures

What You Must Learn – Integration Skills

- How APIs fetch data from databases
- How multiple services integrate safely
- Data validation and consistency
- Error handling in backend systems

Common Mistakes to Avoid

- Writing SQL without normalization
- Hardcoding data instead of querying DB
- No use of triggers or procedures
- Direct DB access between modules
- Poor naming conventions

Final Instructions to Students

Important

- Think beyond your single module
- Design for integration
- Follow DBMS best practices
- Be ready to explain every design choice

Final Takeaway

Think Like System Designers

- Your module is part of a larger system
- Design clean schemas
- Follow naming conventions
- Ensure integration readiness

Evaluation Summary

Pull Requests + Commits together provide a transparent, fair, and scalable evaluation system.